



UFOP

Universidade Federal
de Ouro Preto

Universidade Federal de Ouro Preto – UFOP
Instituto de Ciências Exatas e Biológicas – ICEB
Departamento de Computação – DECOM
Disciplina: Engenharia de Software - BCC 322
Professor: Tiago Garcia de Senna Carneiro

Trabalho Individual - Sprint 1

Alunos: Henrique Ulhôa de Faria, Matheus Mota Gomes

Matrículas: 22.2.4088, 22.2.4101

Turma: 11

Ouro Preto, 17 de maio de 2026

Sumário

Sumário	2
1 CASOS DE USO	3
1.1 Criar um Sistema	3
1.2 Criar um Fluxo	4
1.3 Fluxo sem origem conectado a um sistema	5
1.4 Um Sistema conectado a um Fluxo	6
1.5 Sistema com um fluxo na entrada e outro na saída	7
1.6 Dois sistemas conectados por um fluxo	8
1.7 Um sistema com dois fluxos de saída	9
1.8 Sistema com dois fluxos de saída e um fluxo de entrada	11
1.9 Sistema recebendo varios fluxos de entrada	12
1.10 Sistema com dois fluxos de entrada e um fluxo de saída	14
1.11 Sistema com multiplos fluxos de entrada e saída	15
1.12 Consultar Resultados	17
2 CRITÉRIOS DE ACEITAÇÃO	18
3 CENARIOS DE TESTES	19
3.1 Cenário 1 – Modelo Exponencial	19
3.1.1 Descrição do modelo (Vensim)	19
3.1.2 Pseudo-código	19
3.2 Cenário 2 – Modelo Logístico	20
3.2.1 Descrição do modelo (Vensim)	20
3.2.2 Pseudo-código	20
3.3 Cenário 3 – Modelo Complexo com 5 Sistemas e 6 Fluxos	22
3.3.1 Descrição do modelo (Vensim)	22
3.3.2 Pseudo-código	22
4 DIAGRAMA DE CLASSES	25

1 Casos de Uso

1.1 Criar um Sistema

Ator	Modelador
Descrição	O modelador cria um objeto do tipo <code>System</code> e um modelo.
Pré-condição	Nenhuma.
Fluxo principal	1. O modelador declara um objeto <code>System</code> e <code>Model</code> . 2. O modelo armazena os sistemas e fluxos adicionados. 3. O modelador roda a simulação.
Pós-condição	Um objeto <code>System</code> está disponível para ser associado a um <code>Model</code> .

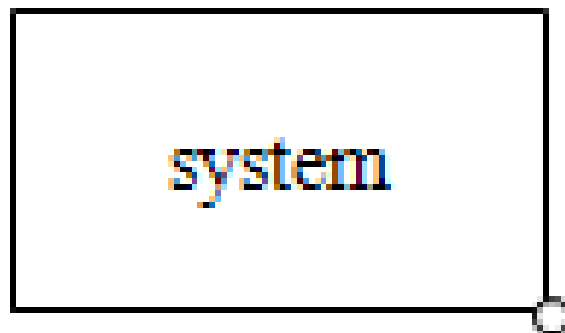


Figura 1 – Diagrama de sequência do caso de uso "Criar um Sistema".

Cenário de teste:

```
1 Model m;  
2 System s1();
```

```

3  assert(s1.getValue() == 0.0);
4  m.add(&s1);
5  assert(m.run(1,100) == 100);

```

1.2 Criar um Fluxo

Ator	Modelador
Descrição	O modelador cria um objeto concreto derivado de MyFlow.
Pré-condição	Nenhuma.
Fluxo principal	1. O modelador instancia uma subclasse concreta de MyFlow. 2. Adiciona o fluxo ao modelo. 3. Roda a simulação.
Pós-condição	Um objeto Flow está pronto para ser adicionado a um Model.

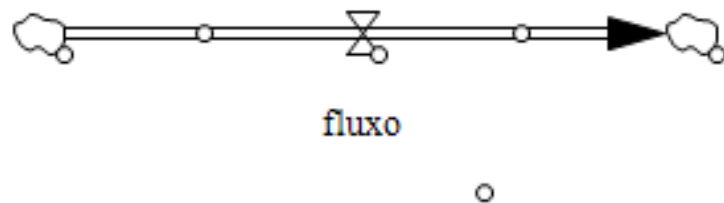


Figura 2 – Diagrama de sequência do caso de uso "Criar um Fluxo".

Cenário de teste:

```

1  Model m;
2  MyFlow f1;
3  m.add(&f1);
4  m.run(1,100);
5  assert(f1.getValue() == 0.0); // fluxo sem sistemas nao altera valor
6  assert(m.run(1,100) == 100); // simula normalmente mesmo com fluxo sem
   sistemas

```

1.3 Fluxo sem origem conectado a um sistema

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e um <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>m.add(system)</code> para cada sistema. 2. Chama <code>m.add(flow)</code> para cada fluxo. 3. Chama <code>f1.setTarget(&s1)</code> para estabelecer a conexão. 4. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

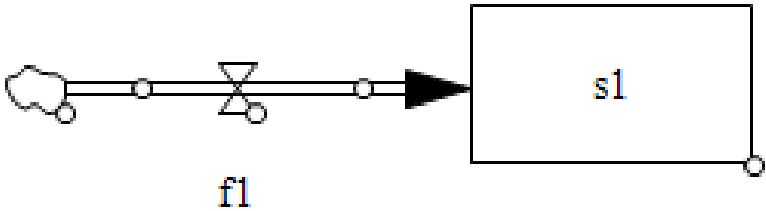


Figura 3 – Diagrama de sequência do caso de uso "Fluxo sem origem conectado a um sistema".

Cenário de teste:

```
1 Model m;  
2 MyFlow f1;  
3 System s1;  
4 m.add(&f1);  
5 m.add(&s1);  
6 f1.setTarget(&s1); // conecta o fluxo ao sistema  
7 assert(m.run(1,100) == 100); // simula normalmente
```

1.4 Um Sistema conectado a um Fluxo

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e um <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>m.add(system)</code> para cada sistema. 3. Chama <code>m.add(flow)</code> para cada fluxo. 4. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

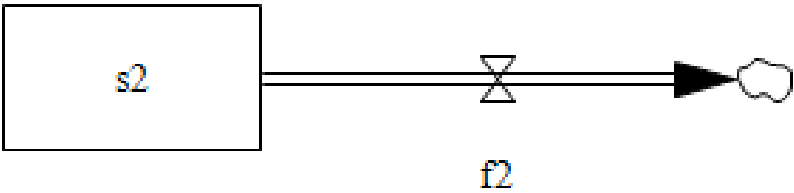


Figura 4 – Diagrama de sequência do caso de uso "Um Sistema conectado a um Fluxo".

Cenário de teste:

```
1 Model m;
2 MyFlow f1;
3 System s1;
4 f1.setSource(&s1); // conecta o fluxo ao sistema
5 m.add(&s1);
6 m.add(&f1);
7 assert(m.run(1,100) == 100); // simula normalmente
```

1.5 Sistema com um fluxo na entrada e outro na saída

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e dois <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para estabelecer a conexão. 2. Chama <code>f2.setSource(&s1)</code> para cada fluxo. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&f1)</code> para cada fluxo. 5. Chama <code>m.add(&f2)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

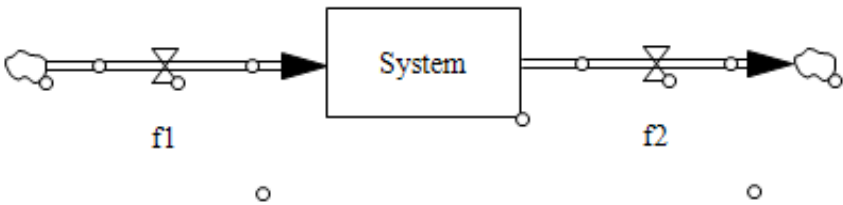


Figura 5 – Diagrama de sequência do caso de uso "Sistema com um fluxo na entrada e outro na saída".

Cenário de teste:

```
1  Model m;  
2  MyFlow f1;  
3  MyFlow f2;  
4  System s1;  
5  f1.setTarget(&s1); // conecta o fluxo ao sistema  
6  f2.setSource(&s1); // conecta o sistema a outro fluxo  
7  m.add(&f1);  
8  m.add(&f2);
```

```

9 m.add(&s1);
10 assert(m.run(1,100) == 100); // simula normalmente

```

1.6 Dois sistemas conectados por um fluxo

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos dois <code>System</code> e um <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>f1.setTarget(&s2)</code> para estabelecer a conexão. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&s2)</code> para cada sistema. 5. Chama <code>m.add(&f1)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

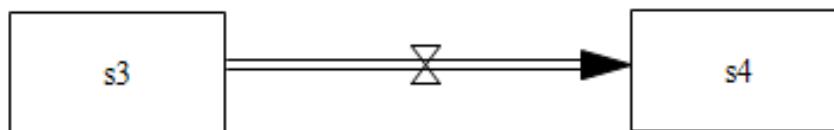


Figura 6 – Diagrama de sequência do caso de uso "Dois sistemas conectados por um fluxo".

Cenário de teste:

```

1 Model m;
2 MyFlow f1("flow", \&s1, \&s2); // fluxo com origem e destino
3 System s1;
4 System s2;
5 m.add(\&s1);
6 m.add(\&s2);
7 m.add(\&f1);

```



```
8 assert(m.run(1,100) == 100); // simula normalmente
```

1.7 Um sistema com dois fluxos de saída

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e dois <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>f2.setSource(&s1)</code> para estabelecer a conexão. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&f1)</code> para cada fluxo. 5. Chama <code>m.add(&f2)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

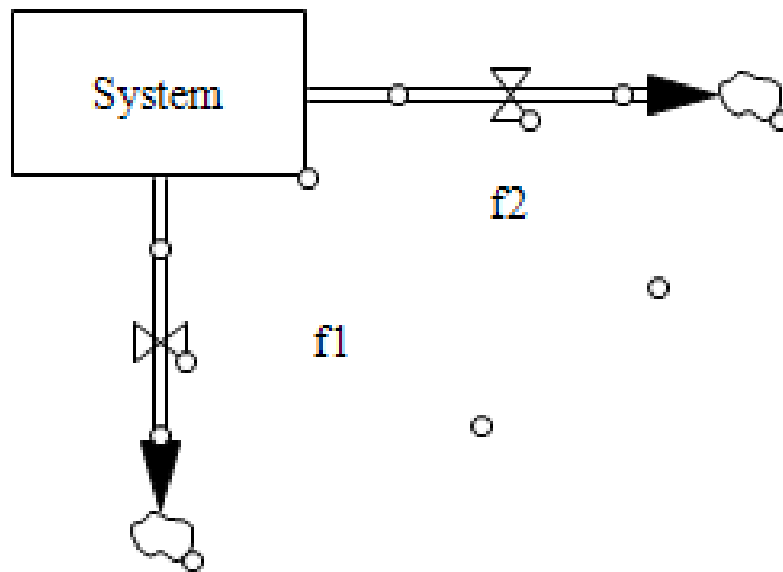


Figura 7 – Diagrama de sequência do caso de uso "Um sistema com dois fluxos de saída".

Cenário de teste:

```

1  Model m;
2  MyFlow f1;
3  MyFlow f2;
4  System s1;
5  f1.setSource(\&s1); // conecta o fluxo ao sistema
6  f2.setSource(\&s1); // conecta o fluxo ao sistema
7  m.add(\&f1);
8  m.add(\&f2);
9  m.add(\&s1);
10 assert(m.run(1,100) == 100); // simula normalmente

```

1.8 Sistema com dois fluxos de saída e um fluxo de entrada

Ator	Modelador
Descrição	O modelador cria um objeto Model e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um System e três Flow já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para cada sistema. 2. Chama <code>f2.setSource(&s1)</code> para cada fluxo. 3. Chama <code>f3.setSource(&s1)</code> para cada fluxo. 4. Chama <code>m.add(&s1)</code> para cada sistema. 5. Chama <code>m.add(&f1)</code> para cada fluxo. 6. Chama <code>m.add(&f2)</code> para cada fluxo. 7. Chama <code>m.add(&f3)</code> para cada fluxo. 8. Roda a simulação.
Pós-condição	O Model contém todos os sistemas e fluxos necessários para a simulação.

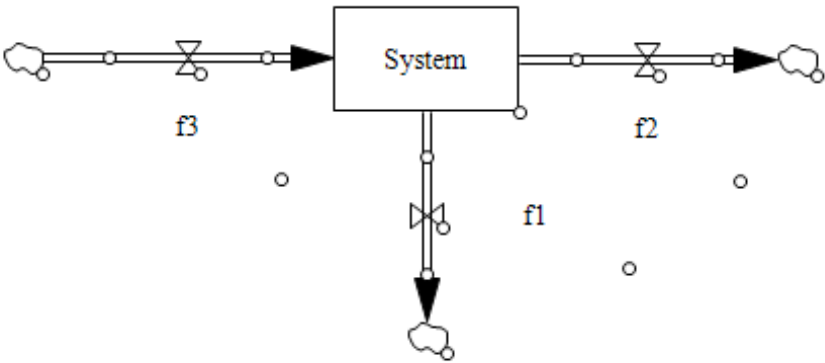


Figura 8 – Diagrama de sequência do caso de uso "Sistema com dois fluxos de saída e um fluxo de entrada".

Cenário de teste:

```

1  Model m;
2  MyFlow f1;
3  MyFlow f2;
4  MyFlow f3;
5  System s1;
6  f1.setTarget(&s1); // conecta o fluxo ao sistema
7  f2.setSource(&s1); // conecta o fluxo ao sistema
8  f3.setSource(&s1); // conecta o fluxo ao sistema
9  m.add(&f1);
10 m.add(&f2);
11 m.add(&f3);
12 m.add(&s1);
13 assert(m.run(1,100) == 100); // simula normalmente

```

1.9 Sistema recebendo varios fluxos de entrada

Ator	Modelador
Descrição	O modelador cria um objeto Model e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um System e dois Flow já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para cada sistema. 2. Chama <code>f2.setTarget(&s1)</code> para estabelecer a conexão. 3. Chama <code>m.add(&f1)</code> para cada fluxo. 4. Chama <code>m.add(&f2)</code> para cada fluxo. 5. Chama <code>m.add(&s1)</code> para cada sistema. 6. Roda a simulação.
Pós-condição	O Model contém todos os sistemas e fluxos necessários para a simulação.

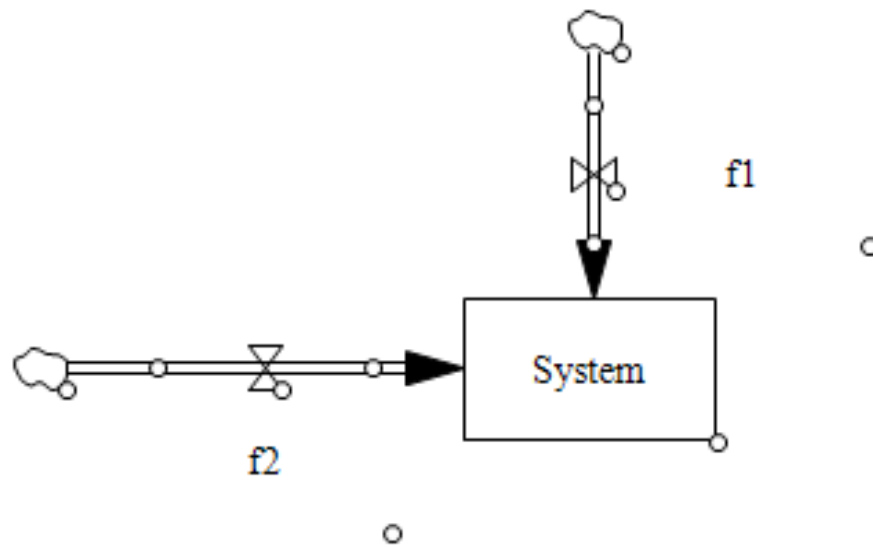


Figura 9 – Diagrama de sequência do caso de uso "Sistema recebendo varios fluxos de entrada".

Cenário de teste:

```

1  Model m;
2  MyFlow f1;
3  MyFlow f2;
4  System s1;
5  f1.setTarget(&s1); // conecta o fluxo ao sistema
6  f2.setTarget(&s1); // conecta o fluxo ao sistema
7  m.add(&f1);
8  m.add(&f2);
9  m.add(&s1);
10 assert(m.run(1,100) == 100); // simula normalmente

```

1.10 Sistema com dois fluxos de entrada e um fluxo de saída

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e três <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para cada sistema. 2. Chama <code>f2.setTarget(&s1)</code> para estabelecer a conexão. 3. Chama <code>f3.setSource(&s1)</code> para estabelecer a conexão. 4. Chama <code>m.add(&s1)</code> para cada sistema. 5. Chama <code>m.add(&f1)</code> para cada fluxo. 6. Chama <code>m.add(&f2)</code> para cada fluxo. 7. Chama <code>m.add(&f3)</code> para cada fluxo. 8. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

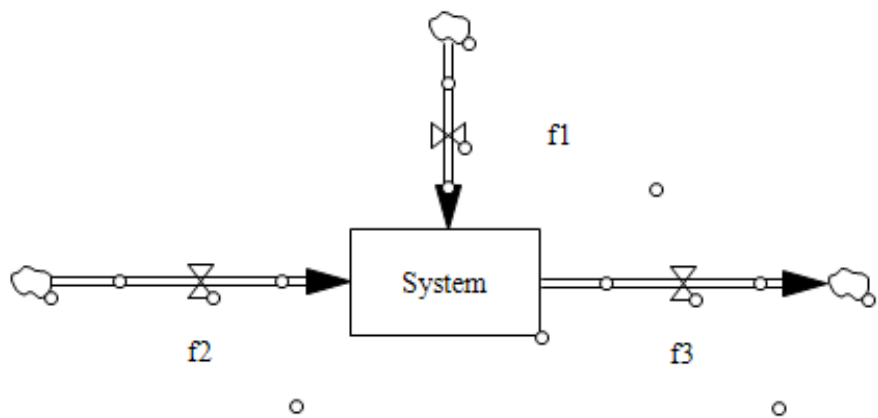


Figura 10 – Diagrama de sequência do caso de uso "Sistema com dois fluxos de entrada e um fluxo de saída".

Cenário de teste:

```
1 Model m;  
2 MyFlow f1;  
3 MyFlow f2;
```

```

4 MyFlow f3;
5 System s1;
6 f1.setTarget(&s1); // conecta o fluxo ao sistema
7 f2.setSource(&s1); // conecta o fluxo ao sistema
8 f3.setSource(&s1); // conecta o fluxo ao sistema
9 m.add(&f1);
10 m.add(&f2);
11 m.add(&f3);
12 m.add(&s1);
13 assert(m.run(1,100) == 100); // simula normalmente

```

1.11 Sistema com multiplos fluxos de entrada e saída

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e quatro <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para cada sistema. 2. Chama <code>m.add(&f1)</code> para cada fluxo. 3. Chama <code>f2.setTarget(&s1)</code> para estabelecer a conexão. 4. Chama <code>m.add(&f2)</code> para cada fluxo. 5. Chama <code>f3.setSource(&s1)</code> para estabelecer a conexão. 6. Chama <code>m.add(&f3)</code> para cada fluxo. 7. Chama <code>f4.setSource(&s1)</code> para estabelecer a conexão. 8. Chama <code>m.add(&f4)</code> para cada fluxo. 9. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

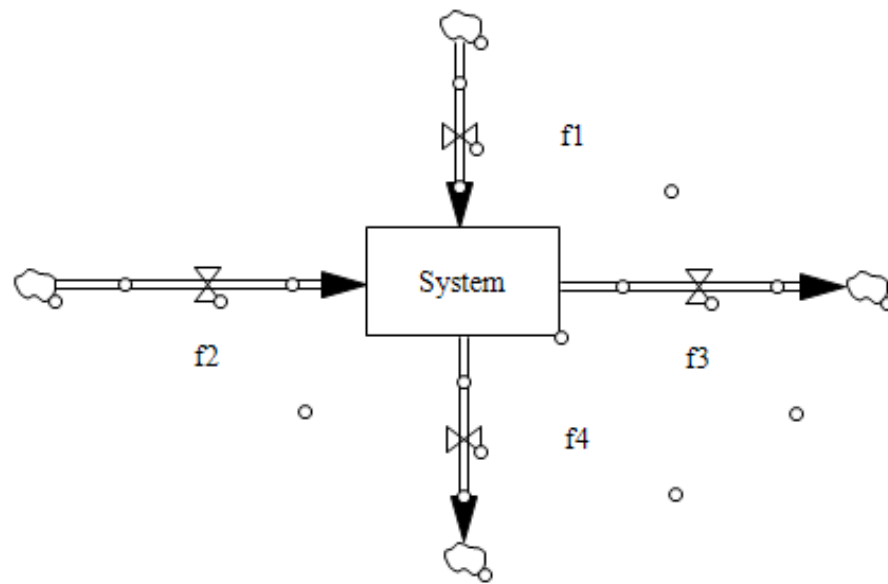


Figura 11 – Diagrama de sequência do caso de uso "Sistema com múltiplos fluxos de entrada e saída".

Cenário de teste:

```

1  Model m;
2  MyFlow f1;
3  MyFlow f2;
4  MyFlow f3;
5  MyFlow f4;
6  System s1;
7  f1.setTarget(&s1);
8  f2.setTarget(&s1);
9  f3.setSource(&s1);
10 f4.setSource(&s1);
11 m.add(&f1);
12 m.add(&f2);
13 m.add(&f3);
14 m.add(&f4);
15 m.add(&s1);
16 assert(m.run(1,100) == 100); // simula normalmente

```


1.12 Consultar Resultados

Ator	Modelador
Descrição	Após a execução da simulação, o modelador consulta os valores finais armazenados nos sistemas.
Pré-condição	A simulação foi executada com sucesso.
Fluxo principal	1. O modelador chama <code>sim.report()</code> ou <code>s.getValue()</code> . 2. Os valores de cada sistema são exibidos.
Pós-condição	Os resultados foram exibidos ou disponibilizados para análise.
Critério de aceitação	Os valores devem ser numericamente iguais aos produzidos pelo Vensim PLE com o arquivo <code>ValidacaoMyVensim.mdl</code> .

Cenário de teste:

```
1 // Após sim.run(tm):  
2 std::cout << "Valor final de P: " << s1.getValue() << std::endl;  
3 sim.report(); // imprime todos os sistemas do modelo
```

2 Critérios de Aceitação

Os critérios de aceitação foram identificados com base nos modelos de referência do enunciado e validados pelo software Vensim PLE.

CA01	Atribuição de Função ao Fluxo
Descrição	O Modelador atribui uma função anônima à equação do fluxo, através da sobrecarga de função.
Verificação	<code>executeEquation()</code> forçara o modelador a implementar a lógica específica para cada fluxo. Devido a existencia da classe abstrata <code>MyFlow</code> .

CA02	Crescimento Exponencial
Modelo	P=10, taxa de natalidade = 0,3, 100 passos de tempo.
Resultado esperado	P cresce exponencialmente sem limite superior, conforme Figura 3(a) do enunciado.
Verificação	P(100) numericamente igual ao valor do Vensim PLE.

CA03	Crescimento Logístico
Modelo	P=10, taxa = 0,3, Pmax=70, 100 passos.
Resultado esperado	P converge para ≈ 70 , conforme Figura 3(b).
Verificação	$ P(100) - 70 < 0,01$ em relação ao Vensim PLE.

3 Cenários de Testes

Os cenários de teste a seguir foram derivados diretamente dos modelos definidos no arquivo `ValidacaoMyVENSIM.mdl`, que serve como referência de validação da API. O arquivo define três grupos de modelos com `INITIAL TIME = 0`, `FINAL TIME = 100` e `TIME STEP = 1`.

Estratégia de projeto da API: cada cenário de teste foi elaborado antes da implementação, guiando o design das classes. A API deve permitir escrever os cenários abaixo de forma direta, sem configurações desnecessárias.

3.1 Cenário 1 – Modelo Exponencial

3.1.1 Descrição do modelo (Vensim)

pop1	INTEG(-exponencial, 100)
pop2	INTEG(exponencial, 200)
exponencial	$0.01 * \text{pop1}$

O fluxo `exponencial` retira de `pop1` e deposita em `pop2` a uma taxa constante de 1% de `pop1` por passo de tempo.

3.1.2 Pseudo-código

```
1 // Cenário 1: Modelo Exponencial
2 // Fonte: pop1/pop2 em ValidacaoMyVENSIM.mdl
3 // taxa = 0.01, pop1_ini = 100, pop2_ini = 200, t = [0, 100]
4 //Classe MyFlow
5 class MyFlow : public Flow {
6 public:
7     MyFlow() : Flow(){}
8     double executeEquation() override {
9         return 0.01 * this->source->getValue(); // exponencial = 0.01
10         * pop1
11     }
12 };
13 //main
14 System *pop1 = new System(100);
15 System *pop2 = new System(200);
```

```

16
17 MyFlow *exponencial = new MyFlow(); // fluxo exponencial
18 exponencial->setSource(pop1);
19 exponencial->setTarget(pop2);
20 Model m;
21 m.add(pop1);
22 m.add(pop2);
23 m.add(exponencial);
24 m.run(0, 100);
25
26 MyFlow *exponencial = new MyFlow(); // fluxo exponencial

```

3.2 Cenário 2 – Modelo Logístico

3.2.1 Descrição do modelo (Vensim)

p1	INTEG(-logistica, 100)
p2	INTEG(logistica, 10)
logistica	$0.01 * p2 * (1 - p2 / 70)$

O fluxo logistica transfere de p1 para p2 com crescimento limitado por Pmax = 70. Conforme p2 se aproxima de 70, o fluxo tende a zero.

3.2.2 Pseudo-código

```

1 // Cenário 2: Modelo Logístico
2 // Fonte: p1/p2 em ValidacaoMyVENSIM.mdl
3 // taxa = 0.01, Pmax = 70, p1_ini = 100, p2_ini = 200, t = [0, 100]
4 //Classe MyFlow
5 class LogisticFlow : public Flow {
6 public:
7     MyFlow() : Flow(){}
8     double executeEquation() override {
9         return 0.01 * this->source->getValue() * (1 - this->source->
10             getValue() / 70); // logistica = 0.01 * p2 * (1 - p2 / 70)
11     }
12 };
13 //main
14 System *p1 = new System(100);
15 System *p2 = new System(200);
16

```

```
17 // logistica = 0.01 * p2 * (1 - p2 / 70)
18 MyFlow *logistica = new MyFlow();
19 logistica->setSource(p1);
20 logistica->setTarget(p2);
21
22 Model m;
23 m.add(p1);
24 m.add(p2);
25 m.add(logistica);
26 m.run(1, 100);
27
28 // Critérios de aceitacao:
29 // p2 deve convergir para Pmax = 70
30 assert(abs(p2.getValue() - 70.0) < 1.0);
31
32 // p1 deve ter decaído (perdeu populacao para p2)
33 assert(p1.getValue() < 100.0);
34
35 // valores numericamente iguais ao Vensim PLE
36 cout << "p1(100) = " << p1.getValue() << endl;
37 cout << "p2(100) = " << p2.getValue() << endl;
```

3.3 Cenário 3 – Modelo Complexo com 5 Sistemas e 6 Fluxos

3.3.1 Descrição do modelo (Vensim)

Sistema	Equação (Vensim)	Valor inicial
Q1	INTEG(v - f - g, 100)	100
Q2	INTEG(f - r - t, 0)	0
Q3	INTEG(g + t - u, 100)	100
Q4	INTEG(u - v, 0)	0
Q5	INTEG(r, 0)	0
Fluxo	Equação	Origem → Des- tino
f	0.01 * Q1	Q1 → Q2
g	0.01 * Q1	Q1 → Q3
r	0.01 * Q2	Q2 → Q5
t	0.01 * Q2	Q2 → Q3
u	0.01 * Q3	Q3 → Q4
v	0.01 * Q4	Q4 → Q1

Este cenário valida a capacidade da API de lidar com múltiplos sistemas e fluxos simultâneos, bem como ciclos entre sistemas (Q4 → Q1 via fluxo v).

3.3.2 Pseudo-código

```
1 // Cenário 3: Modelo Complexo (Q1..Q5)
2 // Fonte: ValidacaoMyVENSIM.mdl
3 // Todos os fluxos: taxa = 0.01, t = [0, 100]
4
5 //Classe MyFlow
6 class LogisticFlow : public Flow {
7 public:
8     MyFlow() : Flow(){}
9     double executeEquation() override {
10         return 0.01 * this->source->getValue(); // todos os fluxos:
11             taxa = 0.01 * origem
12     }
13 };
14
15 //main
16 System *Q1;
17 System *Q2;
18 System *Q3;
```

```

18 System *Q4;
19 System *Q5;
20
21 MyFlow *f = new MyFlow('f', Q1, Q2); // fluxo f
22 MyFlow *g = new MyFlow('g', Q1, Q3); // fluxo g
23 MyFlow *r = new MyFlow('r', Q2, Q5); // fluxo r
24 MyFlow *t = new MyFlow('t', Q2, Q3); // fluxo t
25 MyFlow *u = new MyFlow('u', Q3, Q4); // fluxo u
26 MyFlow *v = new MyFlow('v', Q4, Q1); // fluxo v
27
28 Model m;
29 m.add(Q1);
30 m.add(Q2);
31 m.add(Q3);
32 m.add(Q4);
33 m.add(Q5);
34 m.add(f);
35 m.add(g);
36 m.add(r);
37 m.add(t);
38 m.add(u);
39 m.add(v);
40 m.run(1, 100);
41 // Critérios de aceitacao:
42 // Q1 deve ter decaido (saem f e g, entra apenas v)
43 assert(Q1.getValue() < 100.0);
44
45 // Q5 so recebe (fluxo r), nunca perde -- deve ser maior que 0
46 assert(Q5.getValue() > 0.0);
47
48 // valores numericamente iguais ao Vensim PLE
49 cout << "Q1(100) = " << Q1.getValue() << endl;
50 cout << "Q2(100) = " << Q2.getValue() << endl;
51 cout << "Q3(100) = " << Q3.getValue() << endl;
52 cout << "Q4(100) = " << Q4.getValue() << endl;
53 cout << "Q5(100) = " << Q5.getValue() << endl;

```

Os cenários de teste a seguir foram derivados diretamente dos modelos definidos no arquivo ValidacaoMyVENSIM.mdl, que serve como referência de validação da API. O arquivo define três grupos de modelos com INITIAL TIME = 0, FINAL TIME = 100 e TIME STEP = 1.

Estratégia de projeto da API: cada cenário de teste foi elaborado antes da implementação, guiando o design das classes. A API deve permitir escrever os

cenários abaixo de forma direta, sem configurações desnecessárias.

4 Diagrama de Classes

